



Building a Transparency System for Detained Goods in the Port of Radès

IT 325

March 22, 2025

Youssef Echady Sfaxi

Abstract

The Gavel API is a transparency-focused system designed to manage detained goods and auctions at the Port of Rades in Tunisia. It addresses inefficiencies such as limited interactivity, lack of transparency, and delays caused by outdated customs processes. By digitizing key operations, the API enables real-time tracking, bidding transparency, and interactive filtering of goods, encouraging trust and boosting auction revenue. The report outlines the API's objectives, technical design, deployment strategies, and security measures while discussing its alignment with the Tunisian context and future scalability. Challenges and proposed enhancements, such as machine learning and blockchain integration, are also explored.

Contents

Introduction	1
1 Problem Analysis and Existing Solutions	3
1.1 Problem Context	3
1.2 Similar APIs	4
1.3 Requirements Analysis	4
1.3.1 Functional Requirements	4
1.3.2 Non-Functional Requirements	5
1.3.3 Constraints and Assumptions	5
1.3.4 Key Metrics for Success	5
2 Design and Methodology	6
2.1 Data Models and Database Design	6
2.1.1 Object Models	6
2.1.2 Class Diagram	7
2.2 API Endpoints and Implementation	7
2.2.1 User Management	8
2.2.2 Detained Goods Management	9
2.2.3 Auction System	11
2.3 Frontend Interface	12
3 Implementation and Deployment	13
3.1 Development Environment	13
3.2 Security Aspects	14
3.3 Threats to Validity	14
3.4 Future Improvements	15
Conclusion	16

Introduction

As illustrated in Figure 1 [3], corruption at Tunisian customs is symbolic of the crisis the country is facing. The auction system is inefficient, poorly structured [3], and suffers from collusion between customs officers and bidders, with some goods never reaching auction. Goods remain at the port for an average of 12 days, adding significant costs to shippers [2]. Research [1] shows that reducing customs delays by one day could boost export revenues by \$400 million, or 1% of GDP.

Currently, customs in Tunisia publish detained goods as static PDF documents on their website, this lack of interactivity discourages engagement and lowers sales. Moreover, bidders are unable to see others' bids, resulting in lower bid values. If bidders could view competing bids, they would likely attempt to outbid each other, driving prices higher and increasing auction revenue.

These issues highlight the urgent need for a transparent, efficient system at Rades Port to combat corruption and improve operations.

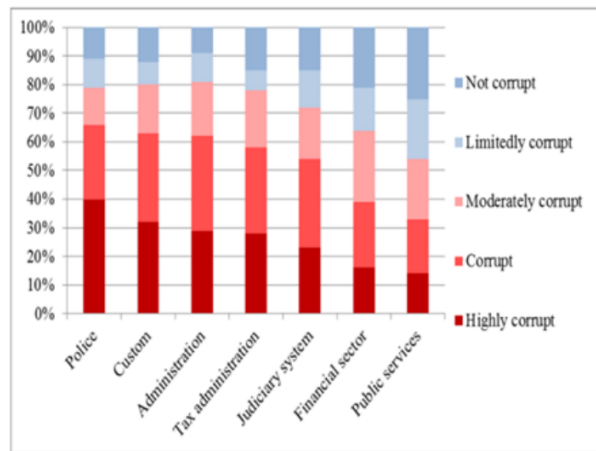


Figure 1: Corruption perception among Tunisian firms (2010)

The Gavel API is designed to address these issues and help manage detained goods and their auctions, providing a streamlined solution for customs officers and users. The system is built with the Tunisian culture and mindset in mind, offering a transparent and interactive interface to reduce inefficiencies. By digitizing and automating key steps in the customs process, the Gavel API eliminates much of the manual paperwork and delays associated with the current system. This reduction in bureaucracy can significantly shorten the time goods spend at the port, minimizing the average 12-day hold period and,

in turn, cutting additional costs for shippers [2].

The Gavel API introduces real-time tracking of goods, enhancing transparency and preventing the mismanagement or loss of goods that often results from corruption and collusion. Additionally, the platform provides an efficient and fair auction system for detained goods. Users can interact with the platform to filter goods, track auctions, and view competing bids, fostering transparency and encouraging competitive bidding. This transparency will help maximize auction revenue while reducing opportunities for bribery and misconduct.

The ultimate goal of the Gavel API is to create a competitive and fair market that reduces shipping costs, increases revenue for the government, and builds trust in the system. If effectively implemented, this system has the potential to significantly contribute to Tunisia's economic recovery by increasing export revenue, promoting trade, and supporting sustainable business practices in the region.

Chapter 1

Problem Analysis and Existing Solutions

1.1 Problem Context

The management of detained goods in Tunisia's customs system faces significant challenges that hinder efficiency, transparency, and profitability. These issues include:

- **Inefficient Information Sharing:** Detained goods are listed in static PDF documents on customs websites, making it impossible for users to filter or search for specific items.
- **Economic Losses:** The current auction system offers no visibility into competing bids. This lack of transparency discourages competitive bidding, resulting in lower final bids and reduced revenue.
- **Corruption and Collusion:** The current system is vulnerable to manipulation and collusion between customs officers and bidders. In some cases, goods never make it to auction, undermining trust and fairness.

Without addressing these challenges, the customs system will continue to impose unnecessary costs, discourage fair trade, and miss critical opportunities to strengthen Tunisia's economic position.

1.2 Similar APIs

Flexport API

Flexport¹ is an API that facilitates the management of imports and exports for clients; from collecting goods at the manufacturer to delivering them to distribution centers and warehouses. While it includes endpoints for retrieving customs entries, its primary focus is on serving clients (exporters and importers), with customs playing a relatively minor role. Additionally, the API lacks functionality for managing auctions.

GovDeals API

GovDeals² is an API used to facilitate online auctions for surplus government properties. This is closer to my API and shares some similarities with it in concept. However, it mainly focuses on government-owned items and not detained goods.

Customs4Trade API

Customs4Trade³ provides customs compliance services, such as classification, documentation, and duty calculations. However, it is focused on European Union markets and lacks flexibility for integration with Tunisian customs processes, such as auction systems for detained goods or addressing corruption.

1.3 Requirements Analysis

1.3.1 Functional Requirements

The system must support multiple user roles with distinct permissions including:

- **Customs officers:** responsible for managing detained goods
- **Bidders:** participating in auctions
- **Owners:** overseeing their goods' status

¹“Flexport API Reference.” 2023. Flexport.com. 2023. <https://apidocs.flexport.com>.

²“GovDeals API.” 2025. Lqdt1.com. 2025. <https://sam.lqdt1.com>.

³“Customs4Trade API.” 2025.Customs4trade.com.2025. <https://api.customs4trade.com>.

A robust goods management feature is also essential, allowing users to add, edit, and remove detained goods, as well as track their life-cycle. Additionally, the auction system must include real-time bidding functionality, where users can see and respond to each other's bids, ensuring transparency and fostering competitive bidding. Search and filtering capabilities are critical to allow users to efficiently locate goods based on criteria such as status. Transparency features, such as the display of bid histories and auction outcomes, are essential to building trust in the system.

1.3.2 Non-Functional Requirements

The platform must deliver high performance, ensuring real-time updates and smooth operation even with a large number of concurrent users. Scalability is another key consideration, as the system must accommodate an increasing number of goods and users over time. Usability is paramount, with an intuitive interface designed for both technical and non-technical users, including local bidders and customs officers unfamiliar with complex systems. Security is a major priority, with measures to protect sensitive data, enforce user authentication, and prevent unauthorized access. Localization features such as support for the Arabic language and compatibility with Tunisian customs procedures are vital.

1.3.3 Constraints and Assumptions

The platform must adhere to Tunisian legal and regulatory requirements regarding customs and auctions. The implementation assumes active collaboration with customs officers and users to ensure its functionality aligns with real-world practices.

1.3.4 Key Metrics for Success

The success of the platform will be measured through several key metrics. These include reducing the average detention period of goods, which currently stands at 12 days, and increasing participation in auctions by fostering a more transparent and efficient process. Competitive bidding visibility is expected to raise the overall bid values, enhancing revenue generation.

Chapter 2

Design and Methodology

2.1 Data Models and Database Design

2.1.1 Object Models

The application organizes its data across five distinct database tables: `role`, `bids`, `goods`, `roles_users`, and `users`.

Role

The `Role` table defines the structure for managing user roles within the system and ensuring role-based access control (RBAC). The class contains an `id` for each role and a `name` representing the role name.

Users

The `UserModel` table represents the structure for managing user accounts within the system. The class includes an `id` for each user, their `name`, `email`, and hashed `password` as well as a boolean value indicating the account status and a `fs_uniquifier` (required by the `Flask-Security` library)

roles_users

The `roles_users` table is an association table used to define a many-to-many relationship between the `UserModel` and `Role` classes. It contains two foreign keys: `user_id` referencing the `users` table, and `role_id` referencing the `role` table

Goods

The `GoodModel` table represents the structure of detained goods within the system. The table includes attributes for product information : `id`, `name`, `category_id`, and `description`), and storage details (`storehouse_id` and `container_id`)

Category

Defines the different categories that detained goods can belong to.

ContainerModel

Contains information about container such as their `id` and `adress`.

Storehouse

Holds information about the storehouse where the detained good is being stored like its `name` and `location`

AutionModel

Represents auctions in the system, auctions will have financial information like the `opening_price`, `guarantee`, as well as the `end_date` of the auction

Bids

The `Bids` table represents the structure of bids within the system. Each bid is linked to a specific user (`user_id`) and a good (`good_id`). The class includes an `id` for each bid, its `value`, the user's `id`, and the good's `id`.

2.1.2 Class Diagram

2.2 API Endpoints and Implementation

This API is divided into 3 functionalities, each with its particular endpoints

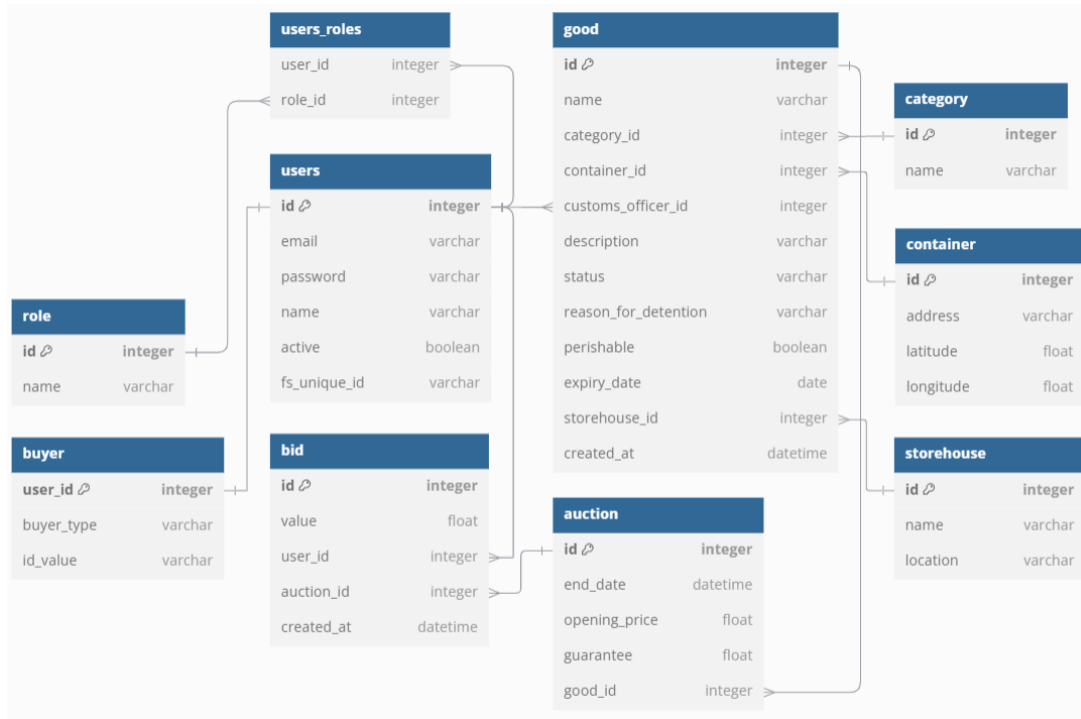


Figure 2.1: Gavel API Class Diagram

2.2.1 User Management

User Signup

User sign up is handled through the `/signup` route and the `POST` method. On sign up the user is required a request with this information:

```

1 {
2   "email": "user@example.com",
3   "password": "string",
4   "role_id": 1,
5   "name": "string",
6   "national_id": null,
7   "fiscal_id": null
8 }
  
```

The `role_id` represents the user's assigned role. A `role_id` of 1 corresponds to customs officers, 2 to buyers, and 3 to owners. National ID and Fiscal ID are only required for buyers (users with a `role_id` equal to 2).

User Login

User login is managed using the `/login` route and the `POST` method. On log in the user is required a request with their email and password:

```
1 {  
2   "email": "user@example.com",  
3   "password": "string"  
4 }
```

User Information

The user (if already logged in) can check their user information (email and role) using the `/user` route and the `GET` method.

2.2.2 Detained Goods Management

Creating new Detained Goods

Adding new detained goods is handled through the `/goods` route and the `POST` method. The user is required: to be a customs officer who is logged in, their given jwt token in the request headers under the `"Authorization"` key, and a request body with this information:

```
1 {  
2   "name": "string",  
3   "container_id": "string",  
4   "status": "string",  
5   "description": null,  
6   "category_id": 0,  
7   "reason_for_detention": null,  
8   "storehouse_id": null,  
9   "perishable": true,  
10  "expiry_date": null,  
11 }
```

Not all fields are required to add a new detained goods, there are only 6 required items: name, container_id, opening_price, status, guarantee, and perishable.

Listing all Detained Goods

To see all detained goods users can use the `/goods` route and the `GET` method. This route doesn't require any authentication or authorization as anyone can see the goods detained by customs. This request will return a `JSON` list containing all of the detained goods. The list will be ordered by priority, with perishable goods placed first, ordered by their expiry date, then non-perishable goods ordered by their due date. Additionally, the goods can be filtered by their status, such as `detained`, `on_sale`, `sold`, or `no_commercial_value`.

Seeing a specific Detained Good

To see a specific detained good, based on its `id`, users can use the `/goods/{goods_id}` path and the `GET` method. This route doesn't require any authentication or authorization as anyone can see the goods detained by customs.

Editing a Detained Good's Data

To edit detained goods that were already added, customs officers can use the `/goods/{goods_id}` path and the `PUT` method. The user is required: to be a customs officer who is logged in, their given `jwt` token in the request headers under the `"Authorization"` key, and a request body with the key-value pairs they want to edit.

Deleting a Detained Good

To delete a detained good, customs officers can use the `/goods/{goods_id}` path and the `DELETE` method. The user is required: to be a customs officer who is logged in, their given `jwt` token in the request headers under the `"Authorization"` key, and the good's `id` in the path URI.

2.2.3 Auction System

Adding a new Auction

To create auctions, customs officers can use the `/goods/{goods_id}/auctions` path and the `POST` method. The user is required: to be a customs officer who is logged in and a request body with auction information:

```
1 {  
2   "opening_price": 0,  
3   "guarantee": 0,  
4   "end_date": "2025-01-29T18:12:38.616Z"  
5 }
```

Placing a Bid on a Detained Good

To place bids, buyers can use the `/goods/{goods_id}/bids` path and the `POST` method. The user is required: to be a buyer who is logged in, their given jwt token in the request headers under the `"Authorization"` key, and a request body with the value of their bid:

```
1 {  
2   "value": 0  
3 }
```

The good has to be on sale (i.e their status has to be set to `on_sale`), and their bid value has to be more than the opening price for that good.

Listing all Bids for a Specific Detained Good

to see a list for a specific detained good, buyers can use the `/goods/{goods_id}/bids` path and the `GET` method. The user is required: to be a buyer who is logged in, their given jwt token in the request headers under the `"Authorization"` key, and the good's id in the path URI. this request returns a list of all bids ordered from highest to lowest

Seeing a Specific Bid

To see details about a specific bid, users can use the `/bids/{bid_id}` path and the `GET` method. The user is required to be logged in, their given jwt token in the request headers

under the "Authorization" key, and the bid's id in the path URI.

Deleting a Bid

To delete their bids, buyers can use the `/bids/{bid_id}` path and the `DELETE` method. The user is required: to be the buyer who posted the bid, their given jwt token in the request headers under the "Authorization" key, and the bid's id in the path URI.

2.3 Frontend Interface

The frontend serves primarily as a documentation and testing interface, with the main focus being the API functionality and its implementation. The frontend consists of:

- Basic static pages rendered using Jinja templates
- Swagger UI integration for interactive API documentation

The front-end was developed using CSS and Bootstrap. The design focuses on user-friendliness and responsiveness, ensuring a seamless experience across different devices. To manage dynamic content, I used Jinja to create adaptable and reusable templates.

Chapter 3

Implementation and Deployment

3.1 Development Environment

Backend Libraries

- **Flask-SQLAlchemy**: ORM for managing database interactions
- **Flask-Security**: For user authentication and authorization
- **Flask-APScheduler**: For background tasks in the app (checking due dates for detained goods)
- **Flask-JWT-Extended**: For managing JWT tokens.
- **Werkzeug**: For password hashing.

Frontend Libraries

- **HTML and CSS**: For building the user interface
- **Bootsreap**: For styling and responsive design
- **Jinja**: For dynamic pages

Utility Libraries

- **UUID**: For generating unique identifiers
- **DateTime**: For managing timestamps and dates

3.2 Security Aspects

The application employs several security measures to ensure the integrity of user data and prevent unauthorized access:

- **Token-Based Authentication:** User authentication is implemented using JSON Web Tokens (JWT). Tokens are issued upon login and verified using the `token_required` decorator for secure access to protected routes. Expired or invalid tokens are explicitly handled, denying unauthorized access.
- **Password Security:** Passwords are hashed using the `pbkdf2:sha256` algorithm from the `werkzeug.security` module before storage, ensuring they cannot be retrieved in plaintext even if the database is compromised.
- **Role-Based Access Control (RBAC):** User roles are managed via a flexible RBAC system, restricting access to routes based on user permissions. The `roles_accepted` decorator enforces these restrictions for specific operations.
- **Input Validation:** Input data is validated using schemas (`UserSignupSchema`, `UserLoginSchema`, and `BidSchema`) to ensure that only valid and secure data is processed. For example, national or fiscal IDs are required for certain roles during user registration.
- **Session Security:** The application uses `login_user` to establish sessions after authentication, combining it with JWT to enhance security during user interactions.

3.3 Threats to Validity

While the Gavel API provides a robust solution for managing detained goods and auctions, several potential threats to its validity should be considered:

- **Security Vulnerabilities:** The API relies on third-party dependencies and external libraries, which may introduce potential security risks. Regular updates and audits are essential to address known vulnerabilities.

- **Scalability Risks:** As user activity increases, the system may face challenges in scaling effectively. Proper load testing and deployment of scalable infrastructure are required to handle peak usage.
- **Human Error:** Errors during data entry or mismanagement by customs officers could compromise data integrity. User training and automated validation checks can minimize such risks.
- **Language Barrier:** The system is designed in English, while customs officers and bidders may be more comfortable using Arabic or French. Localization and multilingual support are needed to address this issue.

3.4 Future Improvements

Machine Learning for Predictions

Machine learning can be utilized to analyze historical auction and bidding data. For example, the system could forecast optimal starting prices for goods, expected final bid values, or the likelihood of a specific good being sold within a given timeframe.

Blockchain for Transparency

Blockchain technology can offer a decentralized and tamper-proof way to manage auction data, enhancing transparency and reducing opportunities for corruption. The challenge however lies in the complexity and cost of integrating blockchain into the limited infrastructure of Tunisia.

Multilingual Support

Supporting multiple languages, including Arabic, French, and English, would cater to Tunisia's linguistic diversity while also accommodating global stakeholders

Conclusion

The Gavel API represents a critical step toward modernizing Tunisia’s customs operations, with a focus on transparency, efficiency, and accessibility. By addressing core issues like corruption, delays, and the lack of interactivity, the system encourages a more competitive auction environment and builds trust among stakeholders. The API’s real-time tracking, search, and bidding features streamline processes, reducing costs and increasing revenue potential. While challenges like scalability, security, and infrastructure limitations remain, future improvements—such as multilingual support, blockchain integration, and machine learning—offer promising solutions. If successfully implemented, the Gavel API could significantly enhance operations at the Port of Rades, serving as a model for customs modernization in Tunisia and beyond.

Bibliography

- [1] Simeon Djankov, Caroline Freund, and Cong S. Pham. Trading on time. *The Review of Economics and Statistics*, 92:166–173, 2010.
- [2] COMETE ENGINEERING. Etude portant sur la performance du port de rades, 2012.
- [3] World Bank Group. Tunisia - systematic country diagnostic (english), 2015. Accessed January 12, 2025.